Iteration 1 Planning Document
Rules + Validation Sub-Team

Ryan Meroniuk Colby Thompson Kyle Naz Prateek

February 26, 2026

Contents

1 Document Metadata **3**

2 Problem Being Addressed **3**

3 Scope and Ownership **3**
3.1 In Scope . . . 3
3.2 Out of Scope . . . 3

4 Validation Architecture Overview **4**

5 Move Processing Flow **4**

6 Rule Categories **5**

7 Validation Interfaces and Contracts **5**
7.1 Request Contract . . . 5
7.2 Response Contract . . . 5
7.3 Events . . . 5

8 Error Codes **6**

9 State Update Behavior **6**
9.1 On Valid Move . . . 6
9.2 On Invalid Move . . . 6

10 Data Model Touchpoints **6**

11 Reliability and Security **7**
11.1 Reliability . . . 7
11.2 Security/Access . . . 7

12 Cross-Team Integration **7**
12.1 Platform Core Integration . . . 7
12.2 Client/UI Integration . . . 7
12.3 Quality + Testing Integration . . . 7
12.4 Integration + Release Integration . . . 8

13 Dependencies **8**
13.1 What We Need From Other Teams 8
13.2 What We Provide To Other Teams 8

14 Milestones **9**
14.1 Iteration 1 (Design and Planning) 9
14.2 Iteration 2 (Implementation Progress) 9
14.3 Iteration 3 (Stability and Demonstration) 9

15 High-Level Task Breakdown **9**

16 Assumptions and Trade-Offs **10**

17 Acceptance Criteria **10**

18 Use Case Descriptions **11**
18.1 UC-RV-01: Validate and Apply a Legal Move 11
18.2 UC-RV-02: Reject an Invalid Move 12
18.3 UC-RV-03: Detect Game Over After Move 13

19 Risks and Mitigations **14**

1 Document Metadata

- Sub-team: `Rules + Validation`
- Date: February 26, 2026
- Authors: Ryan Meroniuk, Colby Thompson, Kyle Naz, Prateek
- Stack Alignment: React + FastAPI + MongoDB + WebSockets

2 Problem Being Addressed

We make sure the game works correctly. Our main job is to build a server-side move checker that works for any game.

- **Problem statement:** The platform needs a consistent backend process that checks every move, keeps turns in order, and returns clear errors when a move is invalid.
- **Why it matters to the platform-first goal:** This shared validation logic is used by all games. It supports reusable, game-agnostic interfaces, stable integration with Platform Core/UI, and testable behavior for Quality + Testing.
- **Failure impact if not handled well:** Illegal moves could be accepted, turns could get out of sync, players could act in the wrong session/state, and the platform would be unreliable for integration, testing, and demos.

3 Scope and Ownership

3.1 In Scope

- Move validation pipeline
- Server-side rule enforcement
- Auth and permission checks for move-related actions
- Validation error states and messages
- Turn checks
- Validation contracts for Platform Core, UI, and testing
- Validation outcome events needed by Platform Core and Client/UI

3.2 Out of Scope

- UI rendering and interaction design: Client/UI
- Lobby lifecycle ownership (create/join/leave orchestration): Platform Core
- End-to-end test harness ownership: Quality + Testing
- CI/CD pipeline ownership and release process: Integration + Release
- Database schema migration and production operations: Platform Core + Integration + Release

4 Validation Architecture Overview

The Rules + Validation design follows a server-authoritative model. Clients send actions, the backend checks rules, security, and state, and only valid moves change the game state.

Platform Core Interface Alignment (Pending)

Platform Core defines a `GameEngine` interface with methods `createInitialState()`, `validateMove()`, `applyMove()`, and `evaluateEnd()`, run by the `GameSessionManager`. Our pipeline will interact with the GameEngine and deliver valid moves, state changes, and clear error messages.

Key Points

- Entry points: `make_move` over WebSocket (`/ws/game/{game_id}`) and optional endpoints for reconnect/resync.
- Data sent to Rules + Validation: `playerId`, `authToken`, `sessionId`/`gameId`, `roomCode` (if used), `move` payload, `requestId`.
- Move payload (game-agnostic): `actionType`, action parameters, target selection/coordinates (if needed), and client timestamp.
- Validation stages: auth check, membership check, game status check, active-player check, rule check, conflict/version check.
- Success path: move accepted, state updated, turn advanced, update broadcast, state saved.
- Failure path: move rejected, no state change, no turn change, error returned with code/message.
- Output contract: normalized `ValidationResult` and standard events (`move_accepted`, `move_rejected`, `turn_changed`, `game_over`, `error`).

5 Move Processing Flow

1. Receive move request.
2. Authenticate/authorize player.
3. Verify player membership in session.
4. Check game status allows moves.
5. Check it is the player's turn.
6. Check the move is legal for the current game state.
7. Apply or reject the move.
8. Send update/error to Platform Core and clients.

Notes

- Idempotency/duplicate handling: duplicate move IDs are ignored; retries return the prior outcome when possible.
- Ordering/version checks: server checks turn number/state version to block stale moves.

6 Rule Categories

- Turn-order rules: only the active player can submit a state-changing action.
- Legal-move rules: action must be allowed for the current game state.
- Game-finished rules: no new moves accepted after the game ends.
- Membership/authorization rules: only authenticated players in the session may act.
- Duplicate/late-message rules: stale or out-of-order actions are rejected with conflict errors.

7 Validation Interfaces and Contracts

7.1 Request Contract

- `sessionId`: target game session identifier
- `playerId`: acting player identifier
- `authToken`: authentication token/session token
- `move`: action payload (game-agnostic envelope + game-specific body)
- `clientVersion`: client-known state/turn version for conflict checks
- `requestId`: unique request identifier for duplicates

7.2 Response Contract

- `isValid`: decision flag
- `errorCode`: machine-readable error code
- `errorMessage`: user-facing or log-facing reason
- `updatedState`: server-approved state snapshot/delta (on success)
- `nextActivePlayerId`: next turn owner (on success)
- `serverVersion`: authoritative state version after processing
- `requestId`: echoed correlation identifier

7.3 Events

- `move_accepted`
- `move_rejected`
- `turn_changed`
- `game_over`
- `error`

Note

These are the validation outcome types that the Rules + Validation pipeline produces. Platform Core (via EventBus/GameSessionManager) broadcasts these events to clients over WebSocket. Rules + Validation returns a `ValidationResult` to Platform Core, and Platform Core handles delivery.

8 Error Codes

Code	Intended Client/UI Behavior
NOT_YOUR_TURN	Player tried to move out of turn; show turn warning.
ILLEGAL_MOVE	Move breaks game rules; show invalid-move reason.
GAME_FINISHED	Move sent after game end; lock input and refresh.
PLAYER_NOT_IN_SESSION	Not part of the session; rejoin/login flow.
STATE_CONFLICT	Stale/duplicate/out-of-order action; resync state.
UNAUTHENTICATED	Missing/invalid auth token; re-authenticate.
UNAUTHORIZED_ACTION	Authenticated but not allowed for action; block action and show permission error.
INVALID_PAYLOAD	Missing required fields; fix payload and retry.

Note

Client/UI shows messages while keeping the code for tests and logs.

9 State Update Behavior

9.1 On Valid Move

- State update: backend applies the move only after all checks pass.
- Turn update: active player moves to the next valid player only after a successful apply.
- Broadcast: updated state and turn change events are sent to all players in the session.
- Persistence: updated state and move history are saved for reconnect/recovery.

9.2 On Invalid Move

- State update: no state change and no turn change.
- Error response: `move_rejected/error` returned with code and reason.
- Retry/recovery: player can send a new valid action; stale clients must resync on `STATE_CONFLICT`.

10 Data Model Touchpoints

- Game status: `Created | WaitingForPlayers | ReadyToStart | Starting | TurnSelect | Ended | ReportingResults`. Moves are only accepted in `TurnSelect`. Other states return `GAME_FINISHED` (if `Ended/ReportingResults`) or an appropirate error.

- Active player: current turn owner ID.
- Session membership: session participant list and roles.
- Board/game payload: authoritative state used for rule checks.
- Move history/version: request IDs, turn numbers, state versions for conflict handling.
- Auth context: token claims/session identity used for authorization checks.

11 Reliability and Security

11.1 Reliability

- Reconnect handling: client reloads latest state via REST and rejoins the WebSocket room.
- Duplicate/late handling: reject or ignore stale/duplicate requests by `requestId` and `serverVersion`.
- Conflict protection: strict server version checks before apply; conflict returns a clear error.

11.2 Security/Access

- Auth requirements: all state-changing actions need a valid token/session.
- Authorization checks: only users in the session can submit `make_move`; non-participants are blocked.

12 Cross-Team Integration

12.1 Platform Core Integration

- Where core calls validation: before any move application and turn change.
- What validation returns: decision, error code/message, updated state, and next player.
- Core dependencies: stable session IDs, participant membership, game lifecycle status.

12.2 Client/UI Integration

- Required UI states/messages: accepted move, rejected move reason, turn indicator, game-over state.
- Required events for turn/error/game-end: `move_accepted`, `move_rejected`, `turn_changed`, `game_over`, `error`.
- UI dependencies: consistent event schema and field names (`event`, `payload`, `timestamp`, `requestId`).

12.3 Quality + Testing Integration

- Test cases needed: valid move, invalid move, not-your-turn, stale state, unauthorized action, finished-game move attempt.
- Mocks/fixtures/contracts provided: error codes, sample payloads, event contract examples.
- QA dependencies: agreed expected outcomes for each error code and state transition.

12.4 Integration + Release Integration

- Contract stability requirements: validation contract for requests/responses/events.
- CI expectations for validation tests: automated unit/integration checks for acceptance/rejection paths.

13 Dependencies

13.1 What We Need From Other Teams

From Platform Core

- API/service: entry point where validation is called.
- Data model: active player, game status, participant membership, session metadata.
- Event routing: publish/subscribe path for accepted/rejected/turn/game-over events.

From Client/UI

- Screens/components: game room UI that consumes validation results.
- Error display contract: mapping from error codes to UI messages.
- State management: client treats server as source of truth and resyncs on conflict.

From Quality + Testing

- Test framework support: integration harness to run validation scenarios end-to-end.
- Shared fixtures: reusable session/move payload fixtures for edge cases.
- Coverage expectations: agreed minimum coverage for key failure modes.

From Integration + Release

- CI/CD pipeline: run validation test suite on merge requests.
- Branch/review rules: cross-team review changes.
- Doc/release standards: location for API/event contract docs.

13.2 What We Provide To Other Teams

To Platform Core

- Validation interfaces: game-agnostic validation entry points and outcomes.
- Error/result contract: stable codes and decision envelope for turn/state decisions.

To Client/UI

- User-facing validation/error codes: documented set of validation failures.
- Events: clear accepted/rejected/turn/game-over behavior for UI state handling.

To Quality + Testing

- Rule test ideas: list of validation scenarios and expected results.
- Edge-case scenarios: duplicate messages, stale versions, unauthorized actions, finished-game actions.

To Integration + Release

- Stable contract docs: maintained request/response/event specs.
- Validation test entry points: commands and fixtures for CI/CD integration tests.

14 Milestones

14.1 Iteration 1 (Design and Planning)

- I1-M1: Finalize problem statement, scope, and cross-team interfaces.
- I1-M2: Complete class diagram, use case package, and state machine aligned with Platform Core flow.
- I1-M3: Define validation error codes and contract draft (request, result, events).
- I1-M4: Complete planning document and presentation with dependencies, risks, and trade-offs.

14.2 Iteration 2 (Implementation Progress)

- I2-M1: Implement baseline validation pipeline with auth, membership, turn, and legality checks.
- I2-M2: Integrate accepted/rejected event flow with Platform Core + Client/UI.
- I2-M3: Add conflict handling and reconnect behavior.
- I2-M4: Provide test ideas and examples to Quality + Testing and pass first integrated flows.

14.3 Iteration 3 (Stability and Demonstration)

- I3-M1: Handle edge cases and reduce integration bugs.
- I3-M2: Ensure validation behavior is stable across all demo game flows.
- I3-M3: Finalize documentation for contracts, errors, and integration behavior.
- I3-M4: Support final showcase with deterministic, test-backed validation outcomes.

15 High-Level Task Breakdown

- `I1-01`: Draft/update Rules + Validation class diagram with interface points.
- `I1-02`: Draft 2–4 use cases and full descriptions (actors, preconditions, flow, postconditions, exceptions).
- `I1-03`: Draft validation/move-processing state machine.
- `I1-04`: Define validation contracts + error taxonomy.

- `I1-05`: Write dependencies map, milestone plan, and risk section.
- `I1-06`: Finalize planning PDF + presentation material.
- `I2-01`: Implement baseline validation pipeline in backend service layer.
- `I2-02`: Integrate event flow with platform core and client.
- `I3-01`: Stabilize edge-case handling and finalize contract docs.

16 Assumptions and Trade-Offs

- Assumption: Rules + Validation runs as server-side logic in the FastAPI backend.
  - Trade-off: improves fairness and consistency, but adds backend complexity and stricter contracts.
- Assumption: Auth checks are part of validation ownership.
  - Trade-off: stronger security control in one pipeline, but depends on upstream auth/session data quality.
- Assumption: Design stays game-agnostic through shared interfaces with pluggable rule adapters.
  - Trade-off: better reuse across games, but adapter design must be disciplined to avoid game-specific bugs.
- Assumption: Client submits actions only; server decides accepted state.
  - Trade-off: prevents client-side desync and cheating, but requires a robust backend.
- Assumption: Conflict protection uses version/request identifiers.
  - Trade-off: safer behavior, but adds complexity to request contracts and debugging.
- Assumption: Event schema stays standardized (`event`, `payload`, `timestamp`, `requestId`).
  - Trade-off: easier integration/testing, but all teams must follow the same schema.

17 Acceptance Criteria

- Validation rejects illegal actions consistently with stable error codes.
- Turn progression advances only after successful validation and apply.
- Auth and session-membership checks are enforced for state-changing actions.
- Duplicate/stale requests do not corrupt state and return deterministic outcomes.
- Cross-team contracts are documented and agreed by dependent teams.
- Iteration 1 deliverables are complete and aligned with the Platform Core plan.

18 Use Case Descriptions

18.1 UC-RV-01: Validate and Apply a Legal Move

Field	Description
Use Case ID	UC-RV-01
Use Case Name	Validate and Apply a Legal Move
Actors	Player; Platform Core (GameSessionManager); Rules + Validation Pipeline
Description	A player submits a move during their turn. The pipeline checks the move and applies it to the game state.

Preconditions

- The game session exists and is in the `TurnSelect` state.
- The player is authenticated with a valid token/session.
- The player is a registered participant in the game session.
- It is currently this player's turn (they are the active player).

Main Success Flow

1. Player sends a `make_move` event over WebSocket with their move payload, `sessionId`, `playerId`, `authToken`, and `requestId`.
2. Pipeline authenticates the player using the provided `authToken`.
3. Pipeline verifies the player is a member of the specified session.
4. Pipeline checks that the game status is `TurnSelect` (game is active and accepting moves).
5. Pipeline confirms the player is the current active player for this session.
6. Pipeline checks the move against the game-specific rules to confirm legality.
7. Pipeline checks `clientVersion`/`requestId` to confirm the move is not stale or duplicated.
8. Move passes all checks. Pipeline applies the move to the authoritative game state.
9. Pipeline evaluates end conditions (win/loss/draw). No end condition detected.
10. Pipeline advances the turn to the next valid active player.
11. Pipeline returns a `ValidationResult` to Platform Core with `isValid: true`, `updatedState`, `nextActivePlayerId`, and incremented `serverVersion`.
12. Platform Core broadcasts `move_accepted` and `turn_changed` events to all session participants.

Postconditions

- The game state has been updated with the applied move.
- The turn has advanced to the next player.
- The move is recorded in the move history with its `requestId` and turn number.
- The updated state is saved for reconnect/recovery.

Exceptions None for this use case. All failure scenarios are covered by UC-RV-02.

18.2 UC-RV-02: Reject an Invalid Move

Field	Description
Use Case ID	UC-RV-02
Use Case Name	Reject an Invalid Move
Actors	Player; Platform Core (GameSessionManager); Rules + Validation Pipeline
Description	A player submits a move that fails one or more checks. The pipeline rejects the move without changing the game state.

Preconditions

- The game session exists.
- The player sends a `make_move` event with a move payload.

Main Success Flow

1. Player sends a `make_move` event over WebSocket with their move payload.
2. Pipeline runs the validation stages (auth → membership → status → turn → legality → conflict).
3. One of the stages fails.
4. Pipeline stops as soon as a stage fails.
5. Pipeline returns a `ValidationResult` to Platform Core with `isValid: false`, the right `errorCode`, and a clear `errorMessage`.
6. Platform Core sends a `move_rejected` event to the player with the error code and message.

Postconditions

- The game state is unchanged.
- The turn does not advance; the same player stays active.
- The `serverVersion` is unchanged.
- The player may submit a corrected move.

Failure Flows

- **Authentication fails:** Player's `authToken` is missing, expired, or invalid. Error code: `UNAUTHENTICATED`.
- **Membership check fails:** Player is not a registered participant in this session. Error code: `PLAYER_NOT_IN_SESSION`.
- **Game status check fails:** Game is in `Ended` or `ReportingResults` state. Error code: `GAME_FINISHED`. Game is in a pre-game state (`Created`, `WaitingForPlayers`, `ReadyToStart`, `Starting`). Error code: `UNAUTHORIZED_ACTION`.
- **Active player check fails:** It is not this player's turn. Error code: `NOT_YOUR_TURN`.
- **Rule legality check fails:** The move breaks game rules (e.g., invalid position, illegal piece movement). Error code: `ILLEGAL_MOVE`.

- **Conflict/version check fails:** The `clientVersion` does not match the current `serverVersion`, or the `requestId` has already been processed. Error code: `STATE_CONFLICT`.
- **Payload validation fails:** Required fields are missing or malformed in the move payload. Error code: `INVALID_PAYLOAD`.

18.3 UC-RV-03: Detect Game Over After Move

Field	Description
Use Case ID	UC-RV-03
Use Case Name	Detect Game Over After Move
Actors	Player; Platform Core (GameSessionManager); Rules + Validation Pipeline
Description	After a valid move is applied, the pipeline checks for a win, loss, or draw and ends the game if needed.

Preconditions

- The game session is in the `TurnSelect` state.
- The player is authenticated, is a session member, and it is their turn.
- The player submits a valid move that passes all checks.

Main Success Flow

1. Player sends a `make_move` event with a valid move payload.
2. Pipeline runs all validation stages (auth, membership, status, turn, legality, conflict). All pass.
3. Pipeline applies the move to the authoritative game state.
4. Pipeline checks the updated state for a win/loss/draw using the game-specific rules.
5. End condition is detected (win for a player, or draw/tie).
6. Pipeline sets the game status to `Ended`.
7. Pipeline returns a `ValidationResult` to Platform Core with `isValid: true`, `updatedState` for the final board, a `gameOver` indicator with the result (winner ID or draw), and game status `Ended`.
8. Platform Core broadcasts `move_accepted` and `game_over` events to all session participants.
9. Platform Core transitions the game to `ReportingResults` and records the outcome for stats/leaderboard.

Postconditions

- The game state reflects the final move and the game status is `Ended`.
- No further moves are accepted for this session (later attempts return `GAME_FINISHED`).
- The game result (winner/draw) is available for Platform Core to record in stats and leaderboards.
- The final state is saved.

Exceptions

- No end condition detected: The game continues normally. Turn advances to the next player as described in UC-RV-01. This is the standard case, not an error.

19 Risks and Mitigations

- **Risk:** Platform Core changes their API, event schema, or `GameEngine` interface during implementation.
  - **Mitigation:** Lock versioned contracts early. Hold regular syncs with Platform Core. Do early integration testing in Iteration 2 to catch mismatches early.
- **Risk:** Game-specific rules are harder to fit into a game-agnostic adapter pattern than expected.
  - **Mitigation:** Prove the adapter design with a simple game (Tic-Tac-Toe) before harder games (Connect Four, Checkers). Keep the adapter interface small.
- **Risk:** Validation pipeline does not integrate cleanly with Platform Core's `GameSessionManager` flow.
  - **Mitigation:** Align on the interface contract before Iteration 2 starts. Document the call sequence and return types. Resolve the pending `GameEngine` alignment (see Section 3).
- **Risk:** Auth and session data from other teams is inconsistent or incomplete, causing false validation failures.
  - **Mitigation:** Add defensive validation with clear error codes so issues are traceable. Do not assume clean input. Log validation stage failures for debugging.
- **Risk:** Scope creep into Platform Core's area (e.g., lobby checks, matchmaking checks, session lifecycle management).
  - **Mitigation:** Keep strict scope boundaries as in Section 2. If a validation concern is outside R+V ownership, document it and escalate to Platform Core rather than absorbing it.